

Reproducing a Kinematic-Wave-Based Adaptive Transit Signal Priority Controller Optimised by a Genetic Algorithm

Original article: Behbahani, H., & Poorjafari, V. (2022). Proposing a kinematic wave-based adaptive transit signal priority control using genetic algorithm. *IET Intelligent Transport Systems*, 17(5), 912–928.

<https://doi.org/10.1049/itr2.12316>

1. Introduction and Article Summary

Adaptive Transit Signal Priority (TSP) seeks to reduce delay for public-transport (PT) vehicles at signalised intersections without unduly penalising general traffic. The reproduced article proposes KATC (a kinematic-wave-based adaptive transit-signal controller) that departs from conventional vehicle-based signal optimisation by minimising average passenger delay across both modes. By weighting each vehicle by its passenger occupancy, a fully loaded bus receives priority proportional to the number of people it carries rather than being treated as a single vehicle.

KATC operates as a closed control loop with four elements: (1) a data layer that reads general-traffic demand and the position and occupancy of approaching PT vehicles, (2) a kinematic-wave delay model that predicts, for any candidate signal plan, the delay of both modes using shockwave analysis, (3) a genetic algorithm (GA) that searches green-time splits and phase sequence to minimise average passenger delay, and (4) an actuation layer that deploys the optimised plan. The signal is re-optimised each cycle, planning over a three-cycle rolling horizon. In its full form (KATC-2), which optimises both green times and phase sequence, the authors report reductions in average passenger delay relative to a vehicle-based SYNCHRO baseline and to a green-time-only variant (KATC-1), with the largest gains at low saturation and high PT occupancy.

This project reimplements the KATC method from scratch and evaluates whether its behaviour and reported improvements can be recreated. Consistent with reproduction studies generally, the objective is faithful reimplementations and an honest account of what reproduces and what does not, rather than an exact match of the original figures. A mathematical model that predicts traffic delay, and a genetic algorithm that searches for the best signal timing. No machine-learning, deep-learning, or reinforcement-learning techniques are involved.

2. The Original Method

The delay model rests on a triangular fundamental diagram, from which queue-formation and queue-dissipation shockwave speeds are derived. For general traffic, queue length during red and the minimum green required to clear it, determine whether a cycle is undersaturated or oversaturated. Undersaturated cycles incur a triangular queue-profile delay, while oversaturated cycles form a residual queue that carries into the next cycle. For PT vehicles, delay is computed by trajectory analysis across two cycles, giving four saturation cases each subdivided by the vehicle's arrival position. The two modal delays are combined into average passenger delay, which serves as the optimisation objective.

The optimiser is a genetic algorithm (GA) operating on a binary chromosome that encodes, per cycle, a two-bit phase-sequence variable and three six-bit coefficient factors distributing green time between major and minor movements. Over a three-cycle horizon the chromosome is sixty bits. The GA is configured with a population of 150, 150 generations, elitism, mask-based crossover, and adaptive mutation rising from 0.01

to 0.02, with fitness defined as the inverse of average passenger delay. Signal plans are constrained so that phase lengths respect the cycle length, a minimum-green floor, and the National Electrical Manufacturers Association (NEMA) dual-ring barrier.

3. Reproduction Methodology

The method was rebuilt as six phases. Phases 1 and 2 form a simulator independent analytical core, validated by 37 automated unit tests. Phases 3 to 6 provide the simulation environment and evaluation.

3.1 Kinematic-wave delay model

The delay model was implemented directly from the article's equations. To confirm accuracy, the equations were transcribed into LaTeX and checked against a rendered comparison of the originals. The article provides no worked numerical example, so the model was validated in two other ways. First, physical-consistency testing confirmed that delays are never negative, that they respond correctly to changes in green time, red time, and demand, and that residual queues form and carry over as expected. Second, that an internal identity holds. The oversaturated queue-length equation reduces exactly to the undersaturated one when the residual vanishes. Because a transcription error in either equation would break this identity, it provides independent evidence that both were reproduced faithfully.

3.2 Genetic-algorithm optimiser

We reproduced the GA using the article's settings and encoding. A separate set of unit tests checked that it worked correctly. These tests confirmed three things. First, a signal plan could be converted to a chromosome and back without any loss. Second, the GA always produced valid plans, even when given extreme input values. Third, it optimised correctly. The best solution never got worse over time, and the same starting seed always gave the same result.

We then connected the optimiser to the delay model and re-ran the article's preliminary test. On a representative scenario, the GA cut average passenger delay by about 46% compared with random signal plans, and it settled on its answer within roughly 25 generations, all in under three seconds (Figure 1). This matches the convergence the article reports. Because this test does not use the simulator, it is an early check that the analytical part of the system works as a whole.

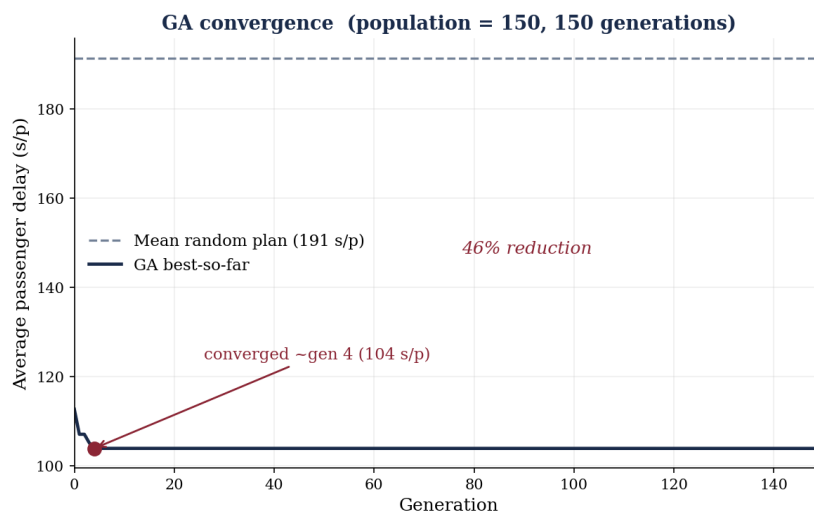


Figure 1. Convergence of the reproduced genetic algorithm on the analytical delay model.

3.3 Simulation environment

The controller was evaluated in SUMO, an open-source microscopic traffic simulator, coupled to the KATC controller through the TraCI interface. A four-legged intersection was constructed with two-lane major and

one-lane minor approaches, each with an exclusive left-turn lane, controlled by a full NEMA dual-ring eight-phase signal. Demand was generated with the article's turning proportions and eight PT routes with their specified headways, scalable by degree of saturation (DOS). Commencing each cycle, the control loop reads traffic state and bus information, runs the KATC optimiser, writes the optimised NEMA plan back to the signal, and runs a cycle, reproducing the read optimise actuate architecture of the original.

3.4 Controllers and experimental design

Three controllers were implemented: (1) KATC-2, which sets both green times and phase sequence, (2) KATC-1, which sets green times only and, (3) a fixed-time baseline. The analytical delay model gives the same result regardless of phase sequence, so on its own, it cannot tell KATC-2 apart from KATC-1. To make the two genuinely different, KATC-2 chooses its phase sequence from the bus delay actually measured in simulation. It does this with an epsilon-greedy strategy: most of the time it uses the option that has performed best so far. But occasionally it tries an alternative to keep learning which is truly best. The experiment follows the original design. Seven saturation levels (DOS 0.4 to 1.0) were crossed with five PT-occupancy levels (20 to 60), across the three controllers, giving 105 one-hour scenarios, each with a two-cycle warm-up.

4. Deviations from the Original

Several deviations from the original were necessary. Each one limits how directly our results can be compared with the articles. They are as follows:

- **Simulator.** The original runs KATC in VISSIM, a commercial simulator. We used SUMO instead, because the free VISSIM trial disables the control interface that KATC depends on. Our aim therefore was to reproduce the method's behaviour and general trends
- **Baseline.** SYNCHRO, a commercial package, was unavailable. A fixed-time Webster-style plan was used as the vehicle-based baseline.
- **Coefficient-factor mapping.** The article does not publish the exact mapping from the three coefficient factors to the eight NEMA movements; a dual-ring consistent interpretation was adopted.
- **Phase sequence.** Phase sequence was resolved from simulation measurements because the analytical model does not depend on it.

5. Results

Table 1 reports the mean average passenger delay for each controller at each DOS level, averaged over PT occupancy. Two observations were made. First, our delay values (roughly 140 to 235 seconds per passenger) are much higher than the article's range of about 35 to 47 seconds per passenger. This gap is expected. SUMO measures delay differently from VISSIM and models driver behaviour differently, so an exact match of the numbers was not expected, after switching simulators. Second, no single controller ranks best across all saturation levels.

Table 1. Mean average passenger delay (s/passenger) by DOS and controller.

DOS	Controller 1	Controller 2	Baseline
0.4	162.7	184.4	146.7
0.5	160.2	185.5	193.4
0.6	165.6	220.6	157.4
0.7	154.7	200.5	144.7
0.8	147.5	140.3	145.0

0.9	154.8	231.9	222.5
1.0	161.3	211.7	166.5

Comparing KATC-1 to KATC-2, KATC-2 achieved lower average passenger delay in 25 of 35 scenarios, a mean improvement of approximately 13.5%. This confirms the article's central claim that adding phase-sequence optimisation on top of green-time optimisation gives a real benefit, which is what sets KATC-2 apart from SYNCHRO and KATC-1.

Analysis against the fixed-time baseline, KATC-2 outperformed fixed-time in 20 of 35 scenarios, but the mean improvement across the grid was slightly negative (approximately -6.4%). This was as a result of two anomalous saturation levels (DOS 0.5 and DOS 0.9) at which KATC-2 performed worse than the baseline. At the remaining five DOS levels, KATC-2 was competitive with or better than fixed-time, with improvements of roughly 5 to 10% at low-to-moderate saturation. Figure 2 shows the improvement of KATC-2 over both comparators as a function of DOS; Figure 3 shows mean delay by controller.

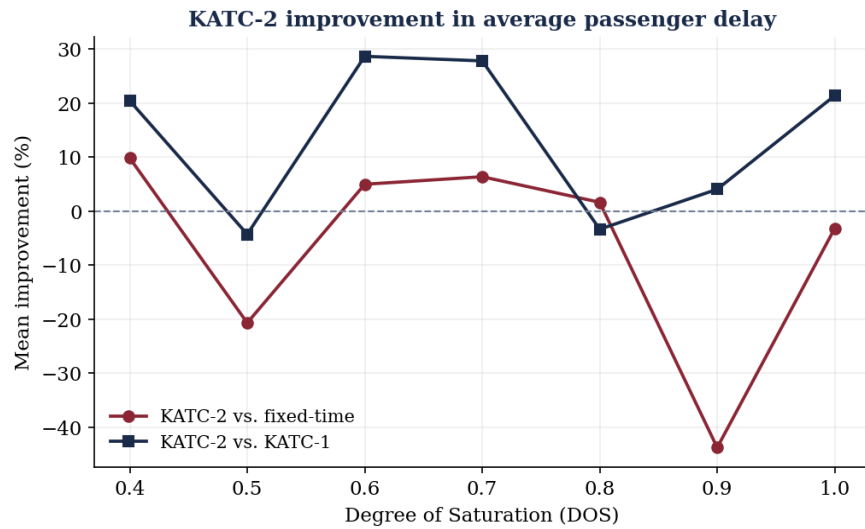


Figure 2. KATC-2 improvement in average passenger delay over fixed-time and over KATC-1, by DOS.

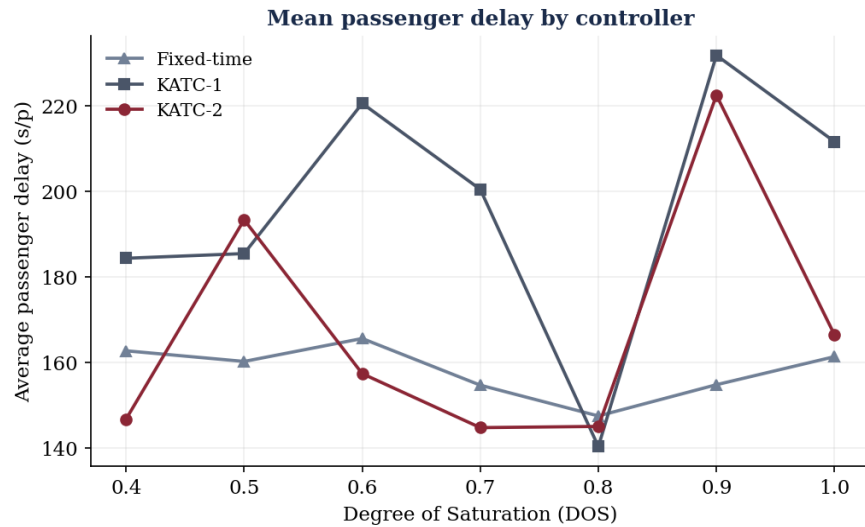


Figure 3. Mean average passenger delay by controller across saturation levels.

6. Discussion and Conclusion

The reproduction worked at the level of method and mechanism. We rebuilt the kinematic-wave delay model and confirmed it against its own internal identity. We reproduced the genetic algorithm faithfully, and it converged just as the article describes. The full read-optimize-actuate loop runs from start to finish in simulation. Most importantly, the article's central claim held up. That claim is that optimising phase sequence gives KATC-2 an edge over KATC-1, and in our runs KATC-2 improved on KATC-1 in the large majority of scenarios.

The reproduction did diverge from the original in two ways. The first is that our delay values came out far higher than the article's. This was expected, since we swapped VISSIM for SUMO and the two simulators measure delay differently. The second is more interesting. KATC-2 did not consistently beat the fixed-time baseline. It won in most scenarios, but the results varied a lot, and there were two points where it lost badly. A few things likely explain this. Because the controller re-optimises from scratch every cycle, it can unsettle the green-time allocation, especially when a steady fixed plan is already close to the best option. On top of that, the delay figures the optimiser aims for is only an estimate, calculated with rough traffic parameters we did not calibrate, so it does not always match what SUMO actually measures. That means the optimiser can pick plans that look good on paper but are not the best in simulation. The two odd saturation levels point to something scenario-specific that would be worth digging into later.

Taken together, the study reproduces the KATC method and its main internal comparison, while also showing that its advantage over a conventional baseline is less reliable once you run it on a different simulator with uncalibrated parameters. That is really where the value of the exercise lies. A reproduction shows understanding not by matching every number, but by working out where a method holds up, where it breaks, and why. Reproducing this work was harder than it needed to be because the original relied on commercial tools, left its coefficient mapping unpublished, and shared no code. Pointing out those gaps is a useful result in its own right.

References

- Behbahani, H., & Poorjafari, V. (2022). Proposing a kinematic wave-based adaptive transit signal priority control using genetic algorithm. *IET Intelligent Transport Systems*, 17(5), 912–928.
<https://doi.org/10.1049/itr2.12316>
- Goldberg, D. E. (1989). *Genetic algorithms in search, optimization, and machine learning*. Addison-Wesley.
- Koonce, P., Rodegerdts, L., Lee, K., Quayle, S., Beaird, S., Braud, C., Bonneson, J., Tarnoff, P., & Urbanik, T. (2008). *Traffic signal timing manual* (Report No. FHWA-HOP-08-024). Federal Highway Administration.
- Lighthill, M. J., & Whitham, G. B. (1955). On kinematic waves II: A theory of traffic flow on long crowded roads. *Proceedings of the Royal Society of London A*, 229(1178), 317–345.
<https://doi.org/10.1098/rspa.1955.0089>
- Lopez, P. A., Behrisch, M., Bieker-Walz, L., Erdmann, J., Flötteröd, Y.-P., Hilbrich, R., Lücken, L., Rummel, J., Wagner, P., & Wießner, E. (2018). Microscopic traffic simulation using SUMO. In *Proceedings of the 21st IEEE International Conference on Intelligent Transportation Systems* (pp. 2575–2582).
<https://doi.org/10.1109/ITSC.2018.8569938>
- Richards, P. I. (1956). Shock waves on the highway. *Operations Research*, 4(1), 42–51.
<https://doi.org/10.1287/opre.4.1.42>
- Webster, F. V. (1958). *Traffic signal settings* (Road Research Technical Paper No. 39). Her Majesty's Stationery Office.

Tools and software

- Anthropic. (2025). *Claude (Opus 4)* [Large language model]. <https://claude.ai>
- Mathpix. (2025). *Mathpix Snip* [Computer software]. <https://mathpix.com>
- uFreeTools. (2025). *LaTeX formula generator* [Computer software]. <https://www.ufreetools.com/en/tool/latex-formula-generator>